JETBRAINS | Kotlin

Asynchronous Programming in Kotlin

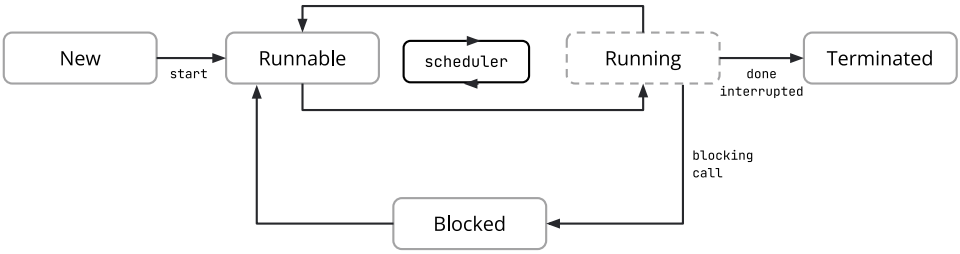
kotlinglang.org/education



What we'll cover

- Parallel and asynchronous programming
- The history of coroutines
- Kotlin coroutines
- Inside CoroutineScope
- Channels
- More

Parallel programming



Parallel programming

- In reality, programs (threads) spend a lot of time waiting for data to be fetched from disk, network, etc.
- The number of threads that can be launched is limited by the underlying operating system (each takes some number of MBs).
- Threads aren't cheap, as they require context switches which are costly.
- Threads aren't always available. Some platforms, such as JavaScript, do not even support them.
- Working with threads is hard. Bugs in threads (which are extremely difficult to debug), race conditions, and deadlocks are common problems we suffer from in multi-threaded programming.
- Threads terminating due to exceptions is a problem that deserves to be a separate point.

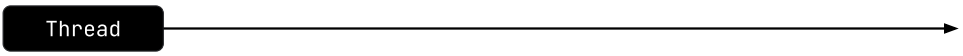
An example

```
fun postItem(item: Item) {
    val token = preparePost()
    val post = submitPost(token, item)
    processPost(post)
}

fun preparePost(): Token { // requestToken
    // makes a request and consequently
blocks the execution thread
    return token
}
```

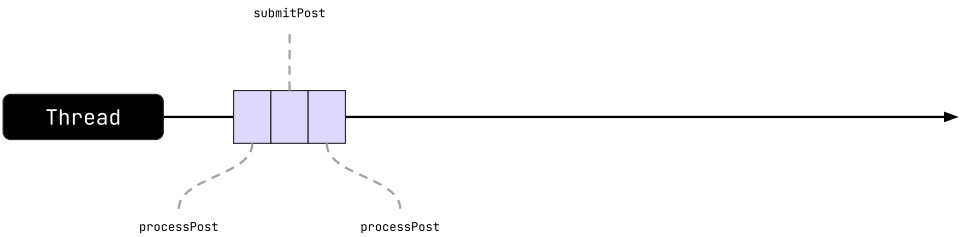
An example

How this code gets executed on a single thread



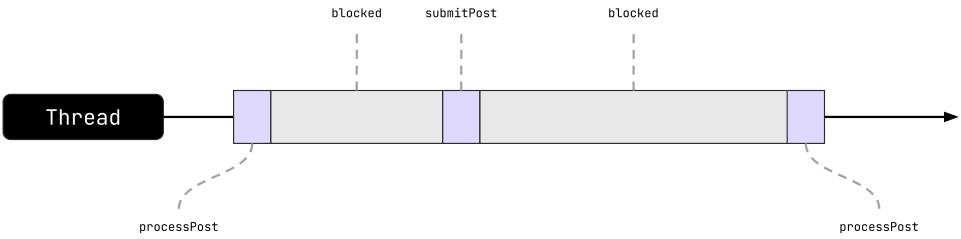
An example

What we want



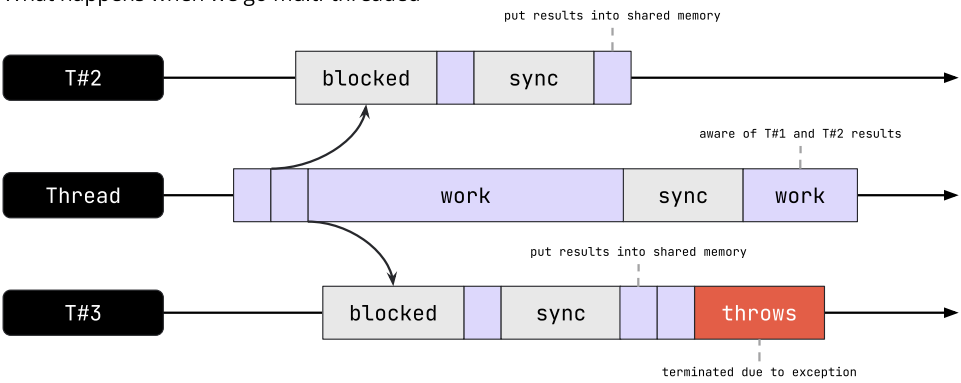
An example

What we get



An example

What happens when we go multi-threaded



Asynchronous Programming

Continuation passing style

```
fun preparePostAsync(callback: (Token) → Unit) {
     // make request and return immediately
     // arrange callback to be invoked later
}
```

With callbacks, the idea is to pass one function as a parameter to another function and have this one invoked once the process has completed.

```
fun postItem(item: Item) {
     preparePostAsync { token →
          submitPostAsync(token, item) { post →
               processPost(post)
          }
     }
}
```

Continuation passing style

```
fun postItem(item: Item) {
     preparePostAsync { token →
           submitPostAsync(token, item) {
                post → processPostAsync(post) {
                      …
                }
          }
     }
}
```

The } ladder is the ~~Stairway to Heaven~~ Highway to “Callback Hell”.

-
- Where is the error handling?
- Callbacks are not asynchronous “by nature”.

Futures, promises, and other approaches

`Promise<T>` encapsulates the callback.

```
fun preparePostAsync(): Promise<Token> {
    // makes request and returns a promise that is completed later
    return promise
}
```

Futures, promises, and other approaches

`Promise<T>` encapsulates the callback.

```
fun postItem(item: Item) {
     preparePostAsync()
           .thenCompose { token →
submitPostAsync(token, item) }
           .thenAccept { post → processPost(post) }
           …
}
```

- This model differs from the typical top-down imperative approach.
- There are different APIs, which vary across libraries, frameworks, and platforms.
- It employs the `Promise<T>` return type instead of the actual we need.
- Each `thenCompute/Accept/Handle` creates a new object.
- Error handling can be complicated.

Kotlin coroutines

```
suspend – a keyword in Kotlin marking suspendable
function.
suspend
fun submitPost(token: Token, item: Item): Post {
      ...
}
```



```
suspend fun postItem(item: Item) {
      val token = preparePost()
      val post = submitPost(token, item)
      processPost(post)
}
```

 marks suspension points in IntelliJ IDEA.

This looks and feels sequential, allowing you to focus on the logic of your code.

The history of coroutines

History and definition

- Melvin Conway coined the term “coroutine” in 1958 for his **assembly** program.
- Coroutines were first introduced as a language feature in Simula’67 with the `detach` and `resume` commands.
- A coroutine can be thought of as an instance of a **suspendable** computation, i.e. one that can suspend at some point and later resume execution, possibly even on another thread.
- Coroutines calling each other (and passing data back and forth) can form the machinery for **cooperative multitasking**.
- Go’09, C#’12, Kotlin’17, C++’20, OpenJDK, Project Loom.

Kotlin

Coroutines came to Kotlin in version 1.1, and they became stable in version 1.3.

- `suspend` – A keyword for marking suspendable functions.
- `kotlin.coroutines` – A tiny part of the standard library.

- `kotlinx.coroutines` – A library with all the necessary functionality. It is not a part of the standard library, meaning there are no additional requirements for the host platform, facilitating multiplatform development.

A coroutine is an instance of suspendable computation. It is conceptually similar to a thread in the sense that it takes a block of code to run and has a similar life-cycle. It is created and started, but it is not bound to any particular thread. It may suspend its execution in one thread and resume in another one. Moreover, like a future or a promise, it can complete with some result (which is either a value or an exception).

Kotlin coroutines

Under the hood

The compiler turns your suspend function:

```
suspend fun submitPost(token: Token, item: Item): Post
{...}
```

Into:

```
fun submitPost(token: Token, item: Item, cont:
Continuation<Post>) {...}
```

Where:

```
public interface Continuation<in T> {
     public val context: CoroutineContext
     public fun resumeWith(result: Result<T>)
}
```

`Continuation<in T>` ~ Generic callback

Under the hood

Code with suspending calls:

```
// code inside postItem
// suspend call 0
val token = preparePost()
// suspend call 1
val post = submitPost(token, item)
// suspend call 2
processPost(post)
```

Is compiled into (simplified version) →

Under the hood

Code with suspending calls:

```
// code inside postItem
// suspend call 0
val token = preparePost()
// suspend call 1
val post = submitPost(token, item)
// suspend call 2
processPost(post)
```

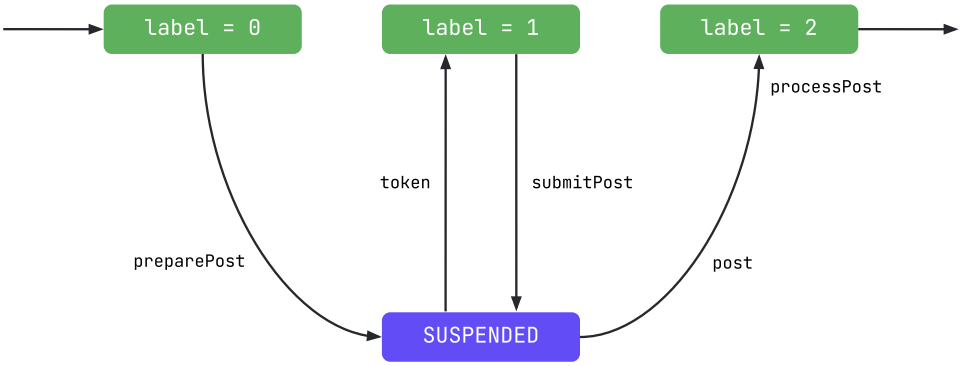
Is compiled into (simplified version) →

~~A large switch???~~ A state machine!

Each label marks a suspension point.

```
// some code here, continuation is created
when(continuation.label) {
     0 → { // suspend call 0
           cont.label = 1;
           preparePost(cont);
     }
     1 → { // suspend call 1
           val token = prevResult;
           cont.label = 2;
           submitPost(token, item, cont);
     }
     2 → { // suspend call 2
           val post = prevResult;
           processPost(post, cont);
     }
}
// more code here
```

State of a coroutine



Practice

Now we can finally post items without blocking the execution thread!

```
fun nonBlockingItemPosting(...) {
    ...
    postItem(item)
}
```

Practice

Now we can finally post items without blocking the execution thread!

```
fun nonBlockingItemPosting(...) {
    ...
    postItem(item)
}
```

The suspending function postItem should be called only from a coroutine or another suspending function.

One cannot just walk into a suspending function.

Inside CoroutineScope

Practice

`suspend` functions can be called from other `suspend` functions or within `CoroutineScope`.

```
fun main() = runBlocking { // this: CoroutineScope
    launch { // launch a new coroutine and continue
        delay(1000L) // non-blocking delay for 1 second (default time unit is ms)
        println("World!") // print after delay
    }
    print("Hello ") // main coroutine continues while the previous one is delayed
}
```

HOFs like `launch` are called *coroutine builders*.

Sophisticated practice

```
val jobs: List<Job> = List(1_000_000) {
     launch(Dispatchers.Default +
CoroutineName("#$it")
          + CoroutineExceptionHandler { context,
error →
               println("$
{context[CoroutineName]?.name}: $error")
          }, // CoroutineContext
          CoroutineStart.LAZY // do not start
instantly
          ) {
               delay(Random.nextLong(1000))
               if (it % 10 == 0) { throw
Exception("No comments") }
               println("Hello from coroutine $it!")
     }
}

jobs.forEach { it.start() }
```

Now we are going to cover all of this step by step.

Scope and context

```
public interface CoroutineScope {
     public val coroutineContext: CoroutineContext
}
```

Easy, isn’t it?

Scope and context

```
public interface CoroutineScope {
     public val coroutineContext: CoroutineContext
}

public interface CoroutineContext {
     public operator fun <E : Element> get(key:
Key<E>): E?
     …

}

public interface Element : CoroutineContext {
     public val key: Key<*>
     ...
}
```

You can think of context like `Map<Key<Element>, Element>`

Inside CoroutineScope

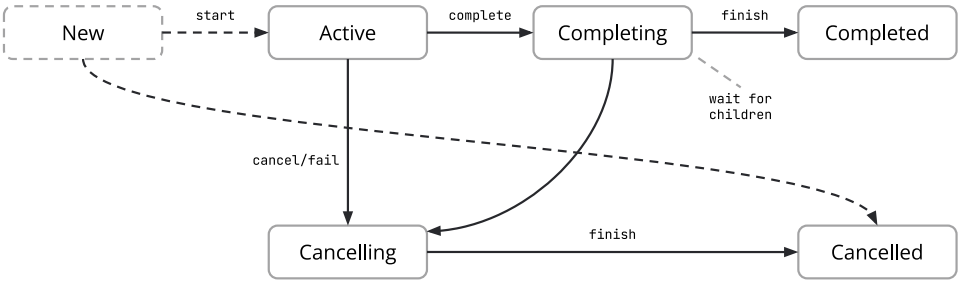
Job

Job

```
public interface Job : CoroutineContext.Element {
     public companion object Key :
CoroutineContext.Key<Job>
     public fun start(): Boolean
     public fun cancel(cause: CancellationException?
= null) public val children: Sequence<Job>
     ...
}
```

- A `Job` is work that is executed in the background.
- It is a cancellable work item with a life-cycle that culminates in its completion.
- Jobs can be arranged into parent-child hierarchies.
- A child's failure immediately cancels its parent along with all its other children. This behavior can be customized using `SupervisorJob`.

Job States


New
start
Active
complete
Completing
finish
Completed
wait for
children
cancel/fail
Cancelling
finish
Cancelled

Job states

state	isAlive	isCompleted	isCancelled
New	false	false	false
Active	true	false	false
Completing	true	false	false
Cancelling	false	false	true
Cancelled	false	true	true
Completed	false	true	false

Inside CoroutineScope

Dispatchers

Dispatchers

```
public abstract class
CoroutineDispatcher : ... {
    ...
    public abstract fun dispatch(context:
CoroutineContext, block: Runnable)
}
```

- `Dispatchers.Default` – A shared pool of background threads, at least 2, depending on the default number of CPU cores. It is an appropriate choice for compute-intensive coroutines.

- `Dispatchers.IO` – A shared pool of on-demand created threads and is designed for offloading IO-intensive blocking operations (such as file/socket IO).

Dispatchers

- `Dispatchers.Main` – A dispatcher that is confined to the `Main` thread operating with UI objects. Usually single-threaded, it is not present in `core`, but is instead provided by packages like `android`, `swing`, etc.
- `Dispatchers.Unconfined` – The unconfined dispatcher should not normally be used in code.
-
-

Private thread pools can be created with `newSingleThreadContext` and `newFixedThreadPoolContext`. (Both are @ExperimentalCoroutinesApi.)

A view of a dispatcher with the guarantee that no more than parallelism coroutines are executed at the same time can be created via:

```
// method of public abstract class
CoroutineDispatcher
@ExperimentalCoroutinesApi
public open fun
limitedParallelism(parallelism: Int):
CoroutineDispatcher
{ ... }
```

Dispatchers

-

An arbitrary `ExecutorService` can be converted into a dispatcher with the `asCoroutineDispatcher` extension function.

```
interface ExecutorService : Executor {
    fun execute(command: Runnable) //
Executor is a SAM with this method
    ...
}

val myExecutorService: ExecutorService
= ...
val myDispatcher =
myExecutorService.asCoroutineDispatcher()
```

A peek under the hood

```
internal class GlobalQueue :
LockFreeTaskQueue<Task>(singleConsumer = false)

internal class CoroutineScheduler(
     @JvmField val corePoolSize: Int, @JvmField val
maxPoolSize: Int,
     @JvmField val idleWorkerKeepAliveNs: Long = ...,
     @JvmField val schedulerName: String = ...
) : Executor, Closeable {
     ...
     val globalCpuQueue = GlobalQueue()

}

val globalBlockingQueue = GlobalQueue()
...
```

A peek under the hood

```
internal class CoroutineScheduler(...) : Executor,
Closeable {
     ...
     val workers = AtomicReferenceArray<Worker?
>(maxPoolSize + 1)

}

fun dispatch(
     block: Runnable,
     taskContext: TaskContext = NonBlockingContext,
     tailDispatch: Boolean = false
) {
     ...
}
```

A peek under the hood

```
internal inner class Worker private constructor() :
Thread() {
     ...
     val localQueue: WorkQueue = WorkQueue()
     var state = WorkerState.DORMANT
     fun findTask(scanLocalQueue: Boolean): Task? {
           // localQueue → globalBlockingQueue
           return task ?: trySteal(blockingOnly =
true)
     }
}
```

Inside CoroutineScope

Coroutines vs threads

Adding contexts

```
val jobs: List<Job> = List(1_000_000) {
     launch(
           BaseContext
           + SupervisorJob()
           + CoroutineName("#$it")
           + CoroutineExceptionHandler { context,
error →
                println("$
{context[CoroutineName]?.name}: $error")
           }, // launch's first argument is
CoroutineContext, which is a sum here
           ...
     ) { ... }
```

- Contexts can be added together. In this case, the rightmost value for a `Key` is taken as the resulting context.
- Since each `Element` implements Coroutine`Context`, this looks like a sum of elements.

Context switching

```
suspend fun preparePost(): Token =
withContext(Dispatchers.IO) { ... }

// submitPost also withContext(Dispatchers.IO)

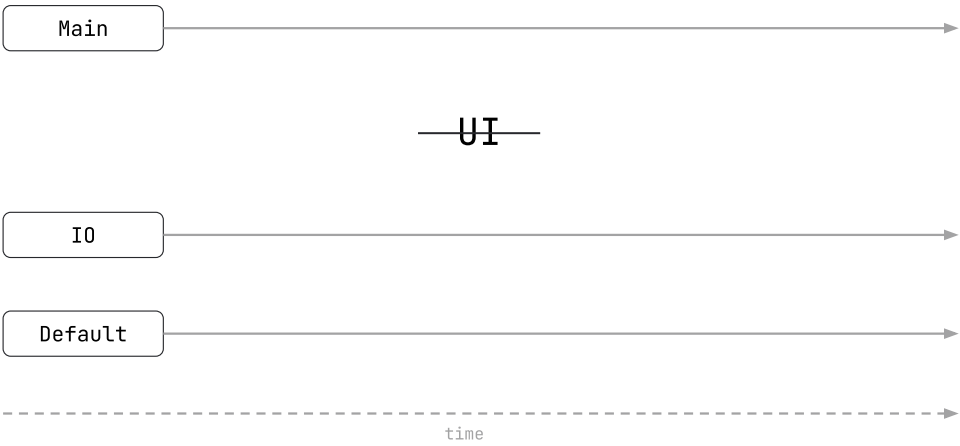
suspend fun processPost(post: Post) =
withContext(Dispatchers.Default) { ... }
```



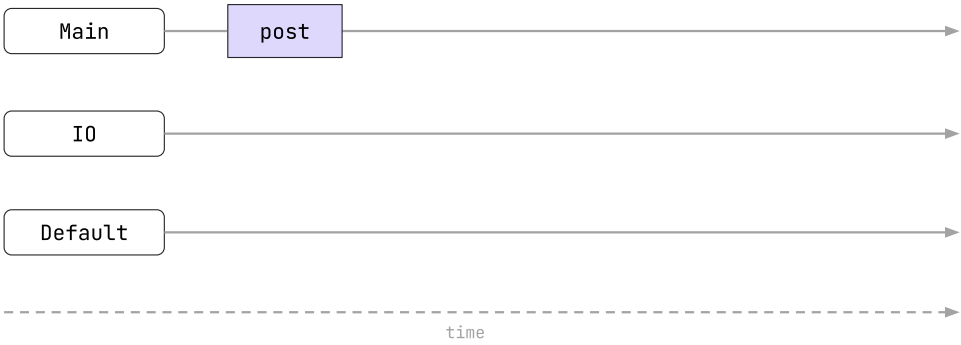
```
suspend fun postItem(item: Item) {
     val token = preparePost()
     val post = submitPost(token, item)
     processPost(post)

// somewhere in our application's code there is a View
and a CoroutineScope related to it
viewScope.launch {
     postItem(someItem)
     // show the result in the UI somehow
}
```

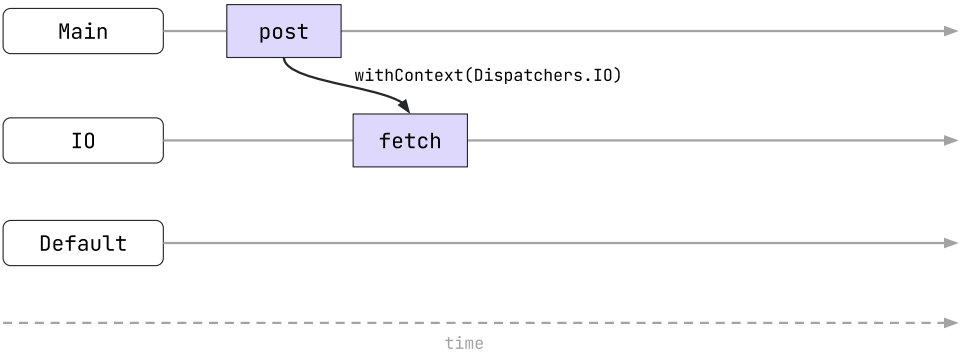
How is this actually better than threads?



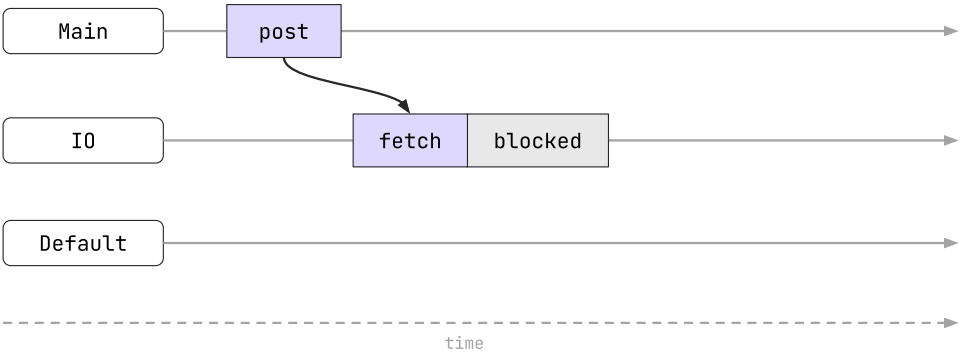
How is this actually better than threads?



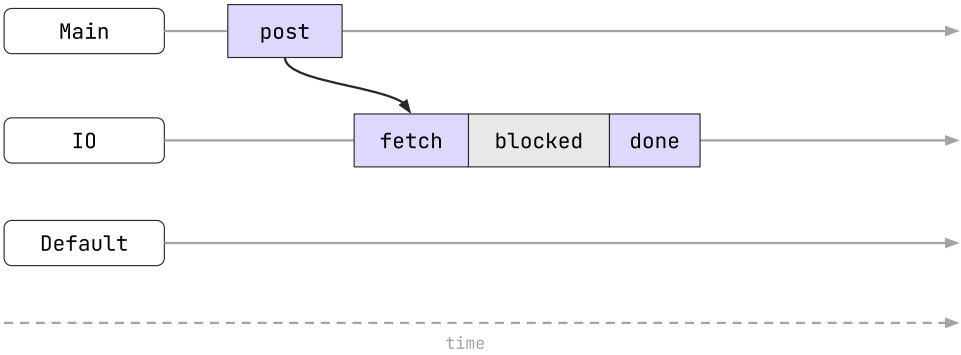
How is this actually better than threads?



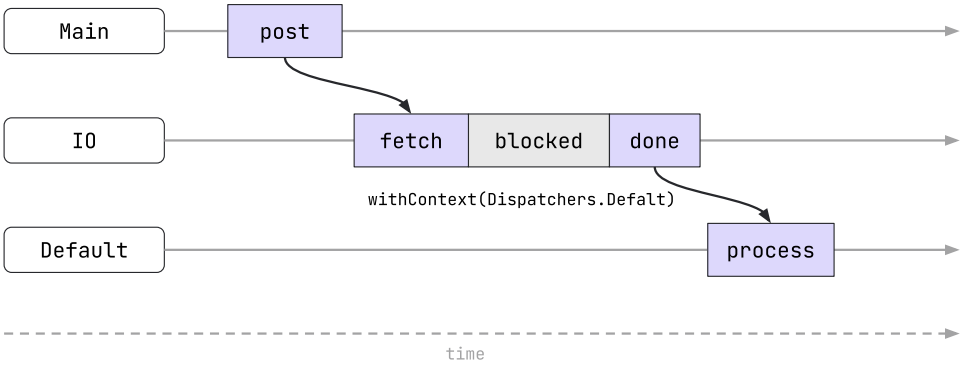
How is this actually better than threads?



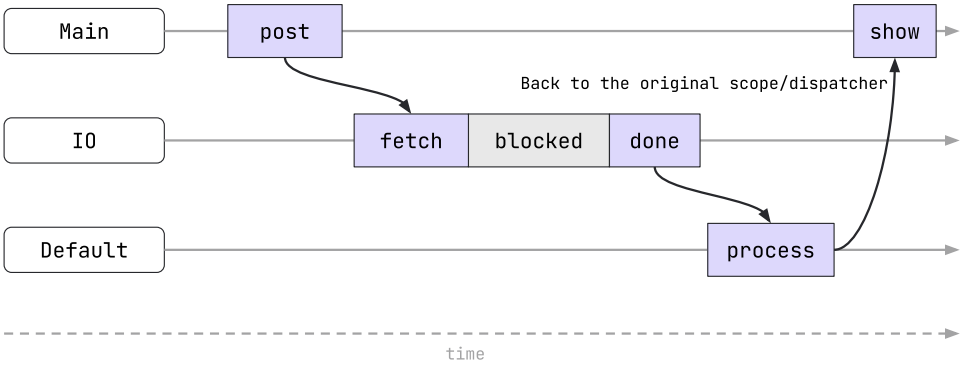
How is this actually better than threads?



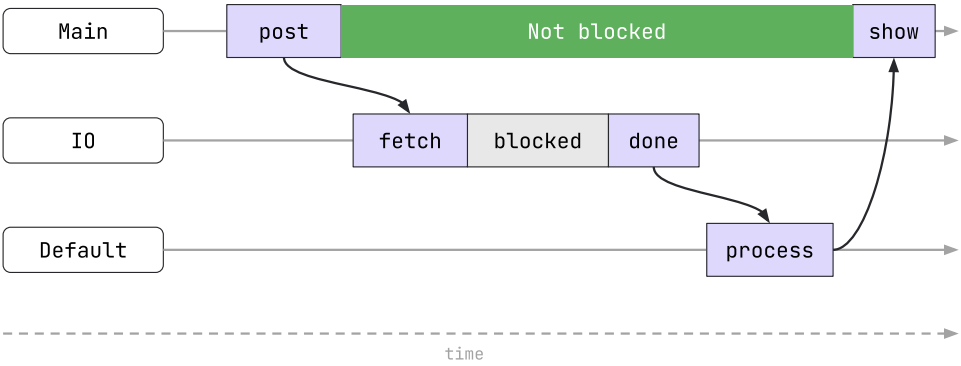
How is this actually better than threads?



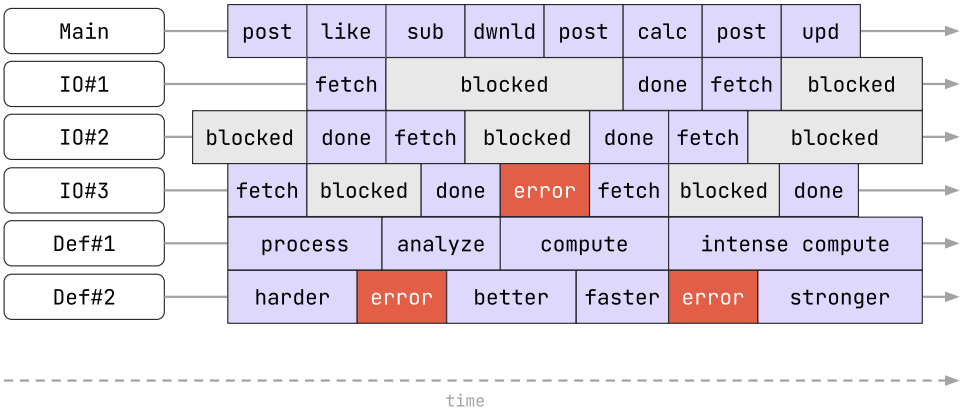
How is this actually better than threads?



How is this actually better than threads?



How is this actually better than threads?



Coroutines - fibers - threads

```
fun main(): Unit = runBlocking {
    repeat(1_000_000) { // it: Int
        delay(Random.nextLong(1000))
        println("Hello from coroutine $it!")
    }
}
```

Coroutines - fibers - threads

```
fun main(): Unit = runBlocking {
    repeat(1_000_000) { // it: Int
        delay(Random.nextLong(1000))
        println("Hello from coroutine $it!")
    }
}
```

WRONG!

The default behavior is sequential, you have to ask for concurrency.

Coroutines - fibers - threads

```
fun main(): Unit = runBlocking {
    repeat(1_000_000) { // it: Int
        launch { // new asynchronous activity
            delay(1000L)
            println("Hello from coroutine $it!")
        }
    }
}
```

Coroutines are like light-weight threads.

Coroutines - fibers - threads

```
fun main(): Unit {
    repeat(1_000_000) { // it: Int
         thread { // new thread
             sleep(1000L)
             println("Hello from thread
$it!")
         }
    }
}
```

Coroutines - fibers - threads

```
fun main(): Unit {
    repeat(1_000_000) { // it: Int
        thread { // new thread
            sleep(1000L)
            println("Hello from thread
$it!")
        }
    }
}
```

Exception in thread "main" java.lang.OutOfMemoryError: unable to create native thread: possibly out of memory or process/resource limits reached.

Coroutines - fibers - threads

```
fun main(): Unit = runBlocking {
    repeat(1_000_000) { // it: Int
        launch { // new asynchronous activity
            delay(1000L)
            println("Hello from coroutine $it!")
        }
    }
}
```

Coroutines are ~~like~~ light-weight ~~threads~~.

Inside Coroutine Scope

Thread switching problem

An important non-guarantee

It is not guaranteed that the coroutine is going to be resumed on the same thread, so be very careful about calling suspending function while holding any monitor.

```
val lock = ReentrantLock()

suspend fun russianRoulette() {
    lock.lock()
    pullTheTrigger()
    lock.unlock()
}
```



Unlock might happen on another thread.

Murphy's law: "Anything that can go wrong will go wrong." Then `unlock` will throw `IllegalMonitorStateException`.

Mutual Exclusion

Mutual Exclusion ⟹ `Mutex`.

```
val mutex = Mutex() // .lock() suspends, .tryLock()
does not suspend
var counter = 0


suspend fun withMutex() {
     repeat(1_000) {
           launch {
                // protect each increment with lock
                mutex.withLock { counter++ }
           }
     }
     println("Counter = $counter") // Guaranteed
`1000`
}
```

Inside CoroutineScope

Exceptions

Exception handling

```
public interface CoroutineExceptionHandler :
CoroutineContext.Element {
     public companion object Key :
CoroutineContext.Key<...>

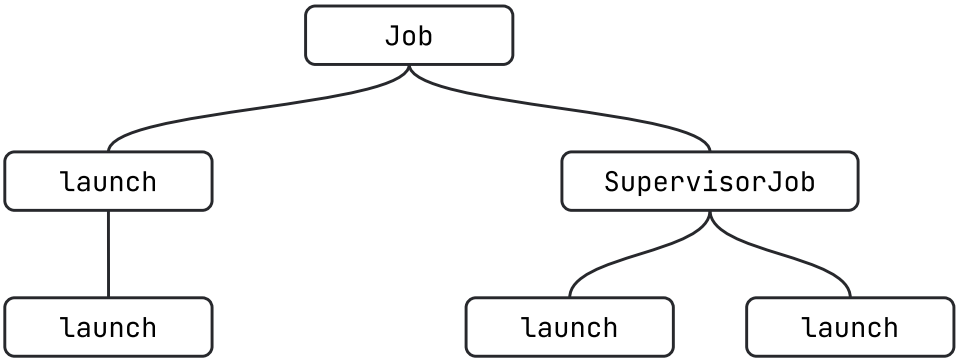
}

public fun handleException(context: CoroutineContext,
exception: Throwable)
```

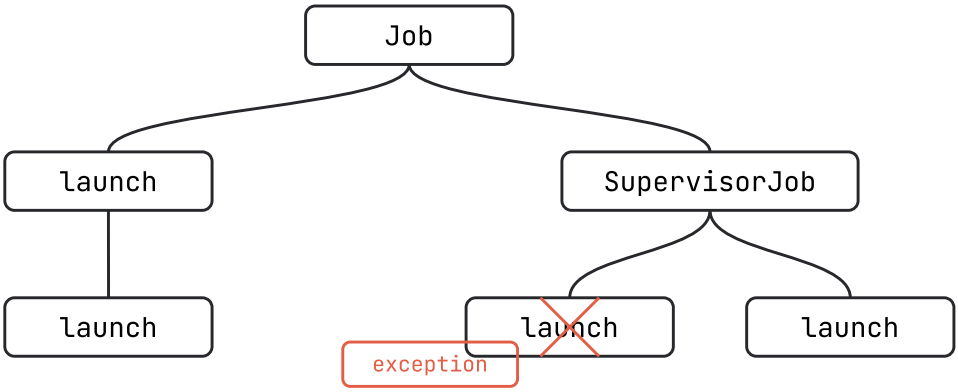
- Children coroutines delegate handling to their parents.
- Coroutines running with `SupervisorJob` do not propagate exceptions to their parents.

- `CancellationExceptions` are ignored.
  - If there is a `Job` in the context, then `Job.cancel` is invoked.
  - *All instances of CoroutineExceptionHandler found via ServiceLoader are invoked.*
  - The current thread's `Thread.uncaughtExceptionHandler` is invoked.

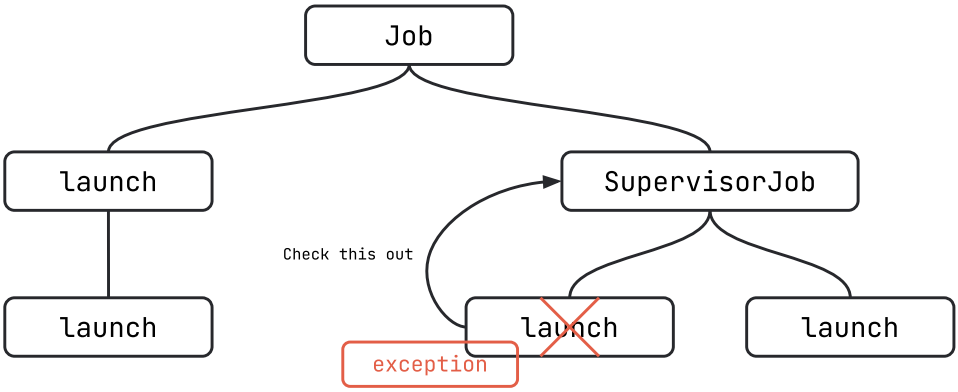
Exception propagation



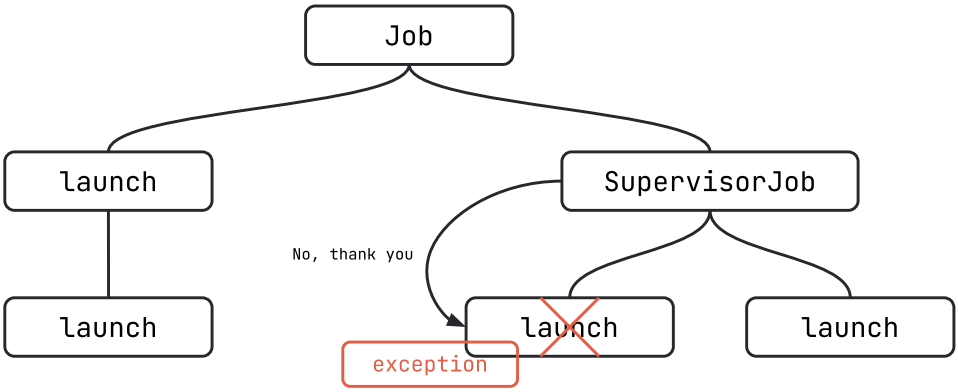
Exception propagation



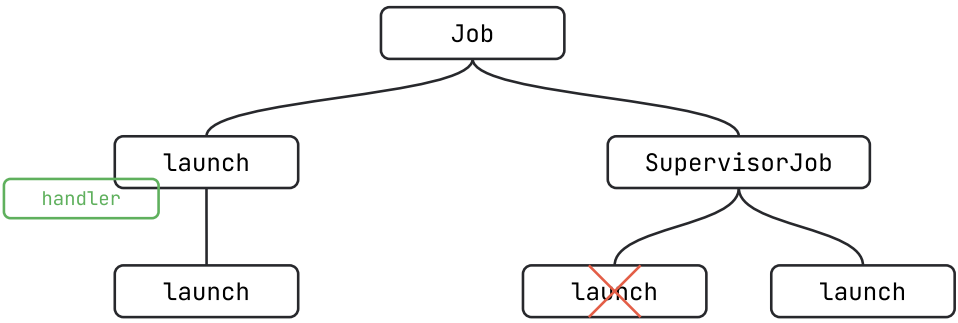
Exception propagation



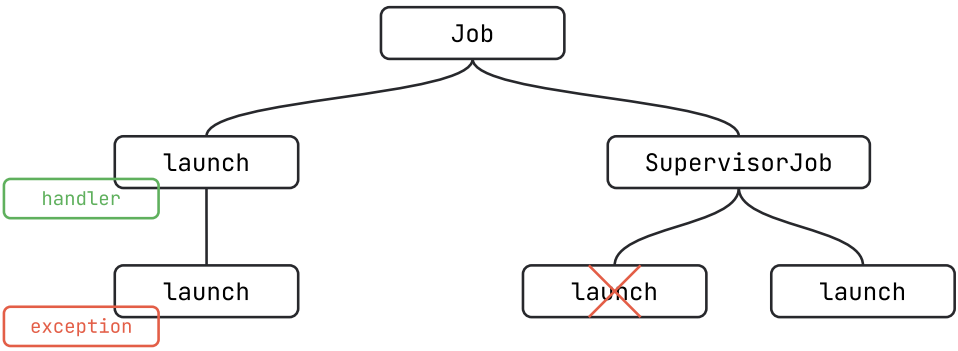
Exception propagation



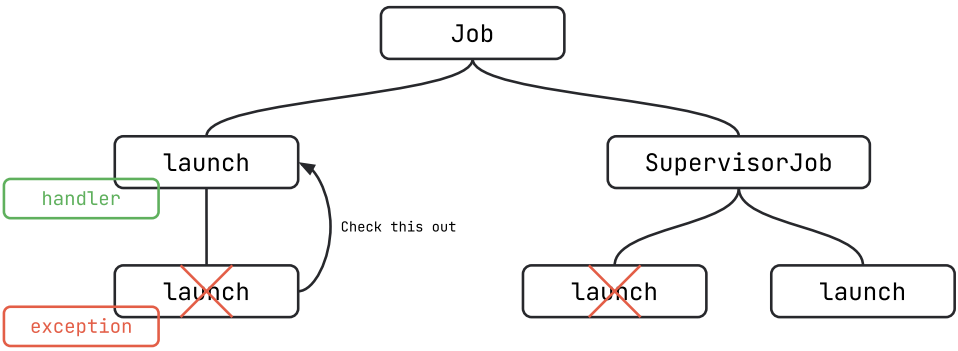
Exception propagation



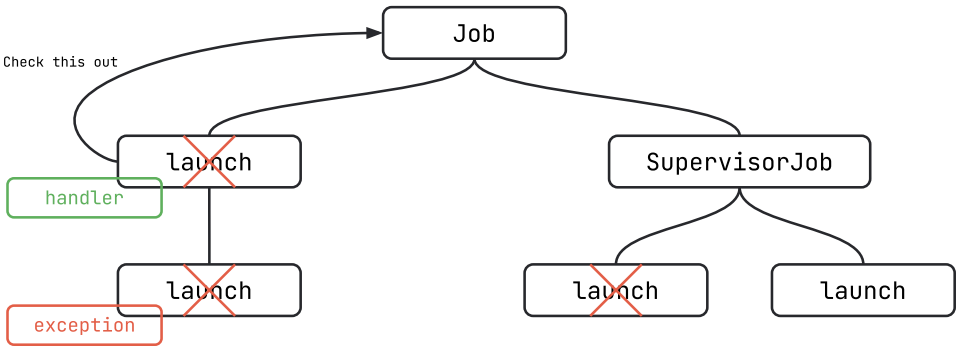
Exception propagation



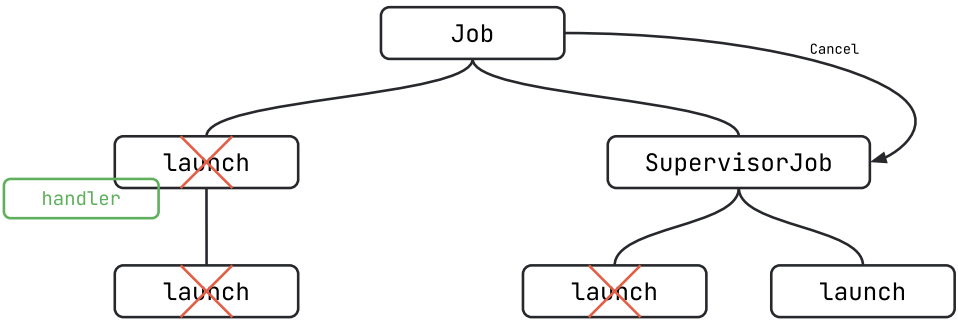
Exception propagation



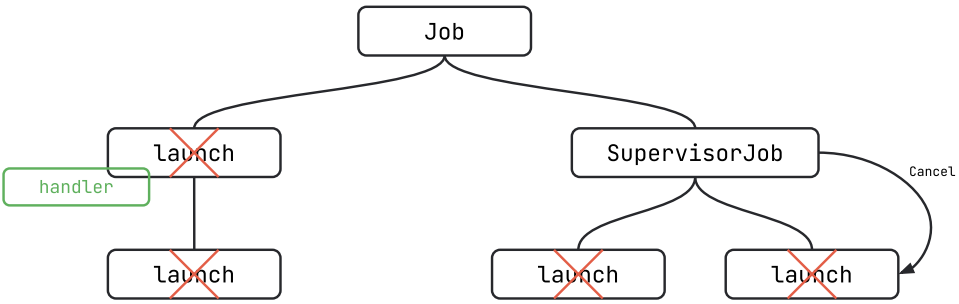
Exception propagation



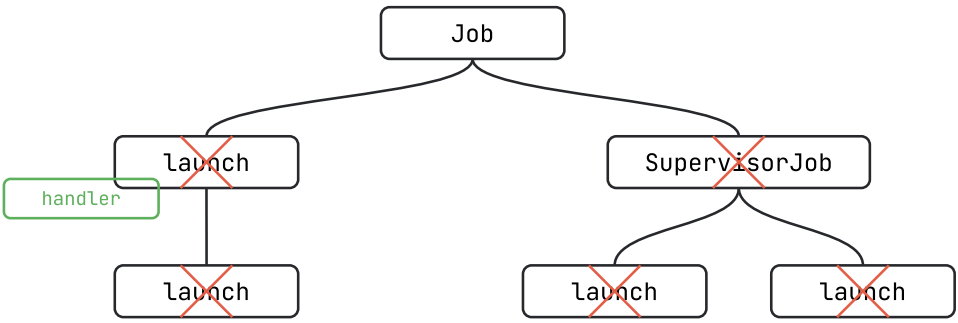
Exception propagation



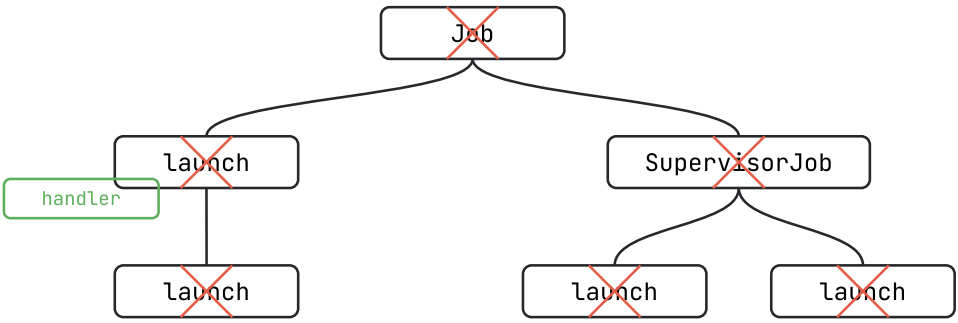
Exception propagation



Exception propagation



Exception Propagation



Now you see it

```
val jobs: List<Job> = List(1_000_000) {
     launch(Dispatchers.Default +
CoroutineName("#$it")
           + CoroutineExceptionHandler { context,
error →
                println("$
{context[CoroutineName]?.name}: $error")
           },
           CoroutineStart.LAZY
           ) {
                delay(Random.nextLong(1000))
                if (it % 10 == 0) { throw
Exception("No comments") }
                println("Hello from coroutine $it!")
     }
}

jobs.forEach { it.start() }
```

This exception handler is useless if this code is not inside a `SupervisorJob`.

Error handling

```
fun main() = runBlocking { // root
coroutine
    val job1 = launch {
        delay(500)
        throw Exception("Some jobs just
want to watch the world burn")
    }
    val job2 = launch {
        println("Going to do something
extremely useful")
        delay(10000)
        println("I've done something
extremely useful")
    }
}
```

Exception in `job1` → propagate to parent → `job2` gets cancelled

Error handling

```
fun main() {
     val scope = CoroutineScope(Dispatchers.Default +
SupervisorJob())
     with(scope) {
           val job1 = launch {
                throw Exception("Some jobs just want
to watch the world burn")
           }
           val job2 = launch {
                delay(3000)

println("I've done something extremely useful")
           }
     }
     scope.coroutineContext[Job]?.let { job →
          runBlocking { job.children.forEach
{ it.join() } }
     } // `job1.join()` will throw, so `it.join()`
should actually be in a `try/catch` block
}
```

Error handling

```
fun main() {
     // `someScope: CoroutineScope` already exists
     someScope.launch { // this coroutine is a child
of someScope
          supervisorScope { // SupervisorJob inside
               val job1 = launch {
                    throw Exception("Some jobs just
want to watch the world burn")
               }
               val job2 = launch {
                    println("Going to do something
extremely useful")
                     delay(3000)
                    println("I've done something
extremely useful")
               }
          }
     }
     ...
}
```



Error handling

```
fun main() {
     val scopeWithHandler =
CoroutineScope(CoroutineExceptionHandler {
          context, error → println("root handler
called")
     })
     scopeWithHandler.launch {
          supervisorScope {
               launch { throw Exception() }
               launch(CoroutineExceptionHandler
{ context, error →
                    println("personal handler
called")
               }) { throw Exception() }
          }
     }
     ...
}
```

Exceptions are not propagated to parents meaning you can override the handler.

Inside CoroutineScope

Structured concurrency

Error handling (revisited)

```
fun processReferences(refs: List<Reference>) {
     for (ref in refs) {
           val location = ref.resolveLocation()
           GlobalScope.launch {
                val content =
downloadContent(location)
                processContent(content)
           }
     }
}
```



- Downloads are launched in the background.
- `GlobalScope` – *This is a delicate API and its use requires care. Make sure you fully read and understand the documentation of any declaration that is marked as a delicate API.* The delicate part is that no `Job` is attached to the `GlobalScope`, making its use dangerous and inconvenient.
- Any `downloadContent` or `processContent` crash results in a coroutines leak.

Structured concurrency

```
suspend fun processReferences(refs: List<Reference>) {
     coroutineScope
{ // new scope with outer context, but a new Job
            for (ref in refs) {
                 val location = ref.resolveLocation()
                 launch { // child of the
coroutineScope above
                      val content =
downloadContent(location)
                      processContent(content)
                 }
            }
     }
}
```

In the event of a `downloadContent` or `processContent` crash, the exception goes to the `coroutineScope`, which stores links to all child coroutines and will cancel them. This is an example of structured concurrency, a concept that is not present in threads.

A helpful convention

This function takes a long time and waits for something:

```
suspend fun work(...) { ... }
```

This function launches more background work and quickly returns:

```
fun CoroutineScope.backgroundWork(...) {
launch { ... }
}
```

or:

```
fun CoroutineScope.moreWork(...): Job = launch { ... }
```

Not:

```
suspend fun CoroutineScope.dontDoThisPlease()
```

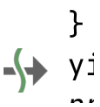
A helpful convention

```
fun
CoroutineScope.processReferences(refs:
List<Reference>) {
    for (ref in refs) {
        val location =
ref.resolveLocation()
        launch { // child of
coroutineScope
            val content =
downloadContent(location)
            processContent(content)
        }
    }
}
```

Inside CoroutineScope

Coroutine cancellation

Cancelling coroutines



```
    val job = launch(Dispatchers.Default) {
        repeat(5) {
            println("job: I'm sleeping $it...")
            Thread.sleep(500) // simulate blocking
work
        }
    yield() // lets the childJob work
    println("main: I'm tired of waiting!")
    job.cancel() // cancels the `job`
    job.join() // waits for `job`'s completion
    println("main: Now I can quit.")
```

- The coroutine (`job`) does not know that somebody is trying to cancel it.
- Cancellation is *cooperative*.

Cancelling coroutines

```
                }
            }
            yield()
```

```
val job = launch(Dispatchers.Default) {
        repeat(5) {
            try {
                println("job: I'm sleeping $it...")
                delay(500)
            } catch (e: CancellationException) {
                println("job: I won't give up $it")
            }
        }

println("main: I'm tired of waiting!")
job.cancelAndJoin() // cancel + join
println("main: Now I can quit.")
```

Cancelling coroutines

```
}
de
```

```
val job = launch(Dispatchers.Default) {
        var i = 0
        while (isActive && i < 5) { // check Job
status
            println("job: I'm sleeping ${i++}...")
            Thread.sleep(500)
        }
delay(1300L)
println("main: I'm tired of waiting!")
 job.cancelAndJoin()
println("main: Now I can quit.")
```

Cancelling coroutines









```
val job = launch {
     try {
          repeat(1_000) {
               println("job: I'm sleeping $it...")
               delay(500L)
          }
     } finally {
          withContext(NonCancellable) {
               println("job: I'm running finally")
               delay(1000L)

println("job: Delayed for 1 sec thanks to
NonCancellable")
          }
     }
}
...
job.cancelAndJoin()
```

Channels

Communicating sequential processes

`Channel` is like `BlockingQueue`, but with suspending calls instead of blocking ones.

- Blocking put → suspending send
- Blocking take → suspending receive
- No shared mutable state!
- Channels are still **experimental**

```
public interface Channel<E> : SendChannel<in E>,
ReceiveChannel<out E> {
     ... suspend fun send(element: E)
     ... suspend fun recieve(): E
     ...
}
```

There are also `trySend` and alike that do not wait.

Practice

```
fun main() = runBlocking {
    val channel = Channel<Int>()
    launch {
        for (x in 1..5)
            channel.send(x * x)
    }
    repeat(5) {
        println(channel.receive())
    }
    println("Done!")
}
```

Prime Numbers



}

```
fun CoroutineScope.numbersFrom(start: Int) =
produce<Int> {
     var x = start
     while (true) send(x++) // infinite stream of
integers from start
```





```
fun CoroutineScope.filter(numbers:
ReceiveChannel<Int>, prime: Int) =
     produce<Int> { for (x in numbers) if (x % prime !
= 0) send(x) }

fun main() = runBlocking {
     var cur = numbersFrom(2)
     repeat(10) {
          println(cur.receive())
          cur = filter(cur, prime)
     }
     coroutineContext.cancelChildren()
}
```

Fan-in and fan-out







```
fun <T> CoroutineScope.production(ch: SendChannel<T>,
msg: T) =
     launch { while (true) {
delay(Random.nextLong(23)); ch.send(msg) } }

fun <T> CoroutineScope.processing(ch:
ReceiveChannel<T>, name: String) =
     launch { for (msg in ch) { println("$name:
received $msg") } }

fun main() = runBlocking {
     val channel = Channel<String>()
     listOf("foo", "bar", "baz").forEach
{ production(channel, it) }
     repeat(8) { processing(channel, "worker #$it") }
     delay(700)

coroutineContext.cancelChildren(CancellationException("Eno
}
```

Details

- Channels are still experimental.
- Channels are fair, meaning that send and receive calls are served in a first-in first-out order.
- By default, channels have `RENDEZVOUS` capacity: no buffer at all. This behavior can be tweaked: The user can specify buffer capacity, what to do when buffer overflows, and what to do with undelivered items.

Select (experimental!)

```
suspend fun selector(
     channel1: ReceiveChannel<String>,
     channel2: ReceiveChannel<String>
): String = select<String> {
     // onReceive clause in select fails when the
channel is closed
     channel1.onReceive { it: String → "b → '$it'" }
     channel2.onReceiveCatching { it:
ChannelResult<String> →
          val value = it.getOrNull()
          if (value ≠ null) {
               "a → '$value'"
          } else {
               "Channel 'a' is
closed" // Select does not stop!
          }
     }
}
```

Miscellaneous

Miscellaneous

Beyond asynchronous programming

Sequences

```
val fibonacci = sequence { // A coroutine builder!
    var cur = 1
    var next = 1
    while (true) {
        yield(cur) // A suspending call!
        cur += next
        next = cur - next
    }
}

val iter = fibonacci.iterator() // nothing happens yet
println(iter.next()) // process up to the first yield
→ 1
println(iter.next()) // wake up and continue → 1
println(iter.next()) // 2 and then to infinity and
beyond
```

Miscellaneous

Under the hood: advanced

Under the hood



Remember this code?

```
suspend fun postItem(item: Item) {
    val token = preparePost()
    val post = submitPost(token, item)
    processPost(post)
}
```

Now that we know much more, let's get a better *approximation* of what's going on under the hood.

Under the hood

```
fun postItem(item: Item, completion: Continuation<Any?
>) {

}

class PostItemStateMachine(
     completion: Continuation<Any?>?,
     context: CoroutineContext?
): ContinuationImpl(completion) {
     var result: Result<Any?> = Result(null)
     var label: Int = 0

}

var token: Token? = null
var post: Post? = null
...
```

Under the hood

```
fun postItem(item: Item, completion: Continuation<Any?
>) {

}

class PostItemStateMachine(...): ... {
     ...
     override fun invokeSuspend(result: Result<Any?>)
{
           this.result = result
           postItem(item, this)
     }
}

val continuation = completion as?
PostItemStateMachine ?:
PostItemStateMachine(completion)
...
```

Under the hood

```
...
when(continuation.label) {
     0 → { ... }
     1 → {
          continuation.token =
continuation.result.getOrThrow() as Token
          continuation.label = 2
          submitPost(continuation.token!!,
continuation.item!!, continuation)
     }
     2 → { ... }
     3 → {
          continuation.finalResult =
continuation.result.getOrThrow() as FinalResult

continuation.completion.resume(continuation.finalResult!!)
     }
     else → throw IllegalStateException(...)
}
...
```

More

Continuation as generic callback

Continuation

Here's a refresher on what `Continuation` looks like:

```
public interface Continuation<in T> {
     public val context: CoroutineContext
     public fun resumeWith(result: Result<T>)
}
```

We are given:

```
suspend fun suspendAnswer() = 42
suspend fun suspendSqr(x: Int) = x * x
```

How can we run `suspendSqr(suspendAnswer)` without `kotlinx.coroutines`?

Continuation

`Continuation` is a generic callback, so we can go back to the continuation passing style:

```
fun main() {
     ::suspendAnswer.startCoroutine(object :
Continuation<Int> {
           override val context: CoroutineContext
                get() = CoroutineName("Empty Context
Simulation")

           override fun resumeWith(result:
Result<Int>) {
                val prevResult = result.getOrThrow()
                ::suspendSqr.startCoroutine(
                      prevResult,

Continuation(CoroutineName("Only name Context")) {
                           it: Result<Int> →
println(it.getOrNull())
                      }
                )
           } // Oh no!
     }) // Closing brackets are coming!
} // Please help! I am being dragged into Callback
Hell!!!
```

Miscellaneous

To wrap existing async code or to implement your own?

To wrap existing async code or to implement your own?

```
suspend fun AsynchronousFileChannel.aRead(b:
ByteBuffer, p: Int = 0) =
     // Scheme: call-with-current-continuation; call/
cc
     suspendCoroutine { cont →
           // CompletionHandler ~ Continuation
           read(b, p.toLong(), Unit, object :
CompletionHandler<Int, Unit> {
                override fun
completed(bytesRead: Int, attachment: Unit) {
                     cont.resume(bytesRead)
                }

}

})

override fun failed(exception: Throwable, attachment:
Unit) {
     cont.resumeWithException(exception)
}
```

To wrap existing async code or to implement your own?

```
fun main() = runBlocking {
     val readJob = launch(Dispatchers.IO) {
           val fileName = ...
           val channel =
AsynchronousFileChannel.open(Paths.get(fileName))
           val buf = ByteBuffer.allocate(...)
           channel.use { // syntactic sugar for `try
{ ... } finally { channel.close() }`
                while (isActive) {
                      ... = it.aRead(buf)
                      ...
                }
           }
     }
     ...
}
```

To wrap existing async code or to implement your own?

```
suspend fun cancellable(…) =
    suspendCancellableCoroutine
{ cancellableCont →

cancellableCont.invokeOnCancellation
{ throwable: Throwable? →
              // release resources, etc.
              ...
         }

}

...

cancellableCont.cancel(…)
```

Miscellaneous

async / await

async / await in Kotlin

```
async Task PostItem(Item item) {
     Task<Token> tokenTask = PreparePost();
     Post post = await SubmitPost(tokenTask.await(),
item);
     ProcessPost();
}
```

- `async` and `await` are keywords in C#.
- Awaiting does not block heavy OS thread.
- `await` is an explicit suspension point.
- `await` is a single function, but depending on its environment it can result in 2 different behaviours.
- The C# approach was a great inspiration for the Kotlin team when they were designing coroutines, as it was for Dart, TS, JS, Python, Rust, C++...

async / await in Kotlin

```
fun
CoroutineScope.preparePostAsync(): Deferred<Token> =
async<Token> { ... }
```



```
suspend fun postItem(item: Item) {
     coroutineScope {
           val token = preparePost().await()
           val post = submitPost(token, item).await()
           processPost(post)
     }
}
```

`Deferred<T> : Job` is a Job that we can get some result from. async is just another coroutine builder. You can write exactly the same in Kotlin!

But why would you do that? This is not idiomatic Kotlin.

async / await in Kotlin



```
  suspend fun postItemAsyncAwait(item:
Item) {
      coroutineScope {
          val deferredToken = async
{ preparePost() }
          // some work
          val token =
deferredToken.await()
          val deferredPost = async
{ submitPost(token, item) }
          // more work
          val post =
deferredPost.await()
          processPost(post)
      }
  }
```

Miscellaneous

Coroutine builders

A zoo of them!

```
public fun CoroutineScope.launch(
     context: CoroutineContext,
     start: CoroutineStart,
     block: suspend CoroutineScope.() → Unit //
suspend lambda
): Job
public fun <T> future(...): CompletableFuture<T> //
jdk8/experimental
public fun <T> CoroutineScope.async(...): Deferred<T>
public fun <T> runBlocking(...): T // Avoid using it
public fun <E> CoroutineScope.produce(
       context: CoroutineContext,
       capacity: Int,
       @BuilderInference block: suspend
ProducerScope<E>.() → Unit
): ReceiveChannel<E>
```

And many more! Like `actor`.

Actor

`Actor` ~ coroutine + channel

```
// Message types for counterActor - Command pattern
sealed class CounterMsg
// one-way message to increment
object IncCounter : CounterMsg() counter
// a request with a reply
class GetCounter(val response: CompletableDeferred<Int>) : CounterMsg()
```

Actor

```
    // This function launches a new counter actor
    fun CoroutineScope.counterActor() =
actor<CounterMsg> {
        var counter = 0 // actor state
        for (msg in channel) { // iterate over
incoming messages
            when (msg) {
                is IncCounter -> counter++
                is GetCounter ->
msg.response.complete(counter)
            }
        }
    }
```

Frequently encapsulated into a separate class.

Miscellaneous

Android

Android

Check out `developer.android.com` to learn how coroutines are used (extensively) in modern Android development.

```
class MyViewModel: ViewModel() {
    init {
        viewModelScope.launch { ... }
    }
}
```

- A `ViewModelScope` is defined for each `ViewModel` in your app.
- A `LifecycleScope` is defined for each `Lifecycle` object.
- `Flow`, which is not covered in this lecture, is common in Android.

Further Reading

- `github.com/Kotlin/KEEP/` – Kotlin design proposals, including coroutines
- `kotlinlang.org` – “Coroutines overview” and “Official libraries/kotlinx.coroutines”
- `github.com/Kotlin/kotlinx.coroutines` – A nicely documented resource
- Roman Elizarov’s talks on YouTube and posts on medium
- `Flow<T>` – Asynchronous Flow
- Kotlin sources at `github.com/JetBrains/kotlin/`
  - All of the code from this presentation can be found in the corountines folder at `github.com/bochkarevko/kotlin-things/`

JETBRAINS | Kotlin

Thank you

